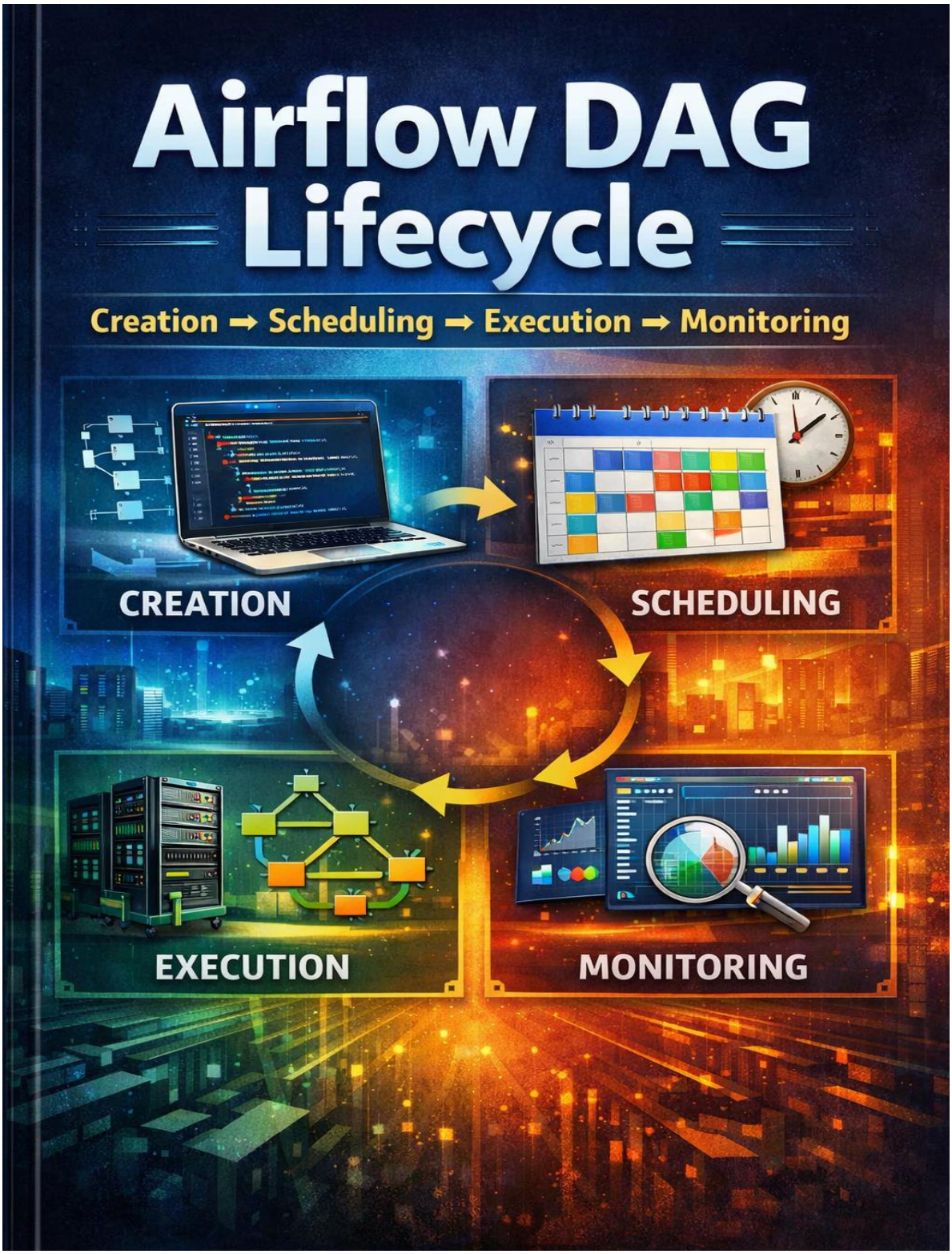

Follow – Sagar Khare

Apache Airflow DAG Lifecycle: Creation → Scheduling → Execution → Monitoring

Executive Summary

Apache Airflow is a leading open-source platform for programmatically authoring, scheduling, and monitoring workflows. At the core of Airflow lies the concept of DAGs (Directed Acyclic Graphs), which represent the complete lifecycle of data pipelines. This document explores the four critical phases of the Airflow DAG lifecycle: Creation, Scheduling, Execution, and Monitoring. Understanding these phases is essential for building scalable, reliable, and maintainable data workflows.

Chapter 1: Understanding DAG Fundamentals

What is a DAG?

A DAG (Directed Acyclic Graph) is a collection of tasks organized to reflect their relationships and dependencies. The term breaks down as follows[1]:

- **Directed**: Tasks flow in a specific direction (e.g., Task A → Task B → Task C)
- **Acyclic**: There are no loops or cycles; a task cannot depend on itself directly or indirectly
- **Graph**: Visual representation of tasks and their dependencies

Key Components of a DAG

Every Airflow DAG consists of these fundamental components[1][2]:

1. **DAG Object**: Represents the entire workflow with metadata and configuration
2. **Tasks**: Individual units of work, instantiated from operators
3. **Dependencies**: Define relationships between tasks (upstream/downstream)
4. **Operators**: Templates for tasks (e.g., BashOperator, PythonOperator, SQLOperator)
5. **Hooks**: Connections to external systems (databases, APIs, cloud services)

Follow – Sagar Khare

6. **Sensors**: Special operators that wait for specific conditions

DAG File Structure

The standard Airflow DAG file contains four major sections:

1. Import Statements

from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime

2. DAG Definition

with DAG(
dag_id='example_dag',
start_date=datetime(2025, 2, 18),
schedule_interval='@daily',
catchup=False,
tags=['production']
) as dag:
# 3. Task Definition
task_1 = BashOperator(
task_id='task_1',
bash_command='echo "Task 1"'
)

```
task_2 = BashOperator(
    task_id='task_2',
    bash_command='echo "Task 2"'
)

# 4. Task Dependencies
task_1 >> task_2
```

Chapter 2: DAG Creation Phase

Best Practices for DAG Design

Creating robust DAGs requires adherence to fundamental principles that ensure reliability and maintainability[2][3]:

Idempotency

Idempotent DAGs produce the same results when re-run with identical inputs. This is critical for Airflow's recovery mechanisms:

- Design tasks to produce the same outcome on every re-run
- Use UPSERT instead of INSERT for database operations to avoid duplicates
- Read from and write to specific partitions, never from "latest available" data
- Avoid using `datetime.now()` for critical computations; use `execution_date` instead

Task Independence

Each task should be treated as a database transaction:

- Never produce incomplete results (e.g., partial data in S3/HDFS)
- Ensure proper error handling and logging
- Use deterministic logic rather than random operations
- Implement proper state management

Configuration Management

Use `default_args` to manage repetitive parameters and avoid errors[2]:

```
default_args = {
'owner': 'data_team',
'retries': 3,
'retry_delay': timedelta(minutes=5),
'start_date': datetime(2025, 2, 18),
'email': ['alerts@company.com'],
'email_on_failure': True
}

with DAG('optimized_dag', default_args=default_args) as dag:
# tasks inherit default_args
pass
```

DAG Versioning and Evolution

Modern Airflow (3.0+) introduces DAG versioning capabilities through AIP-65 and AIP-66[3]:

- **DAG History**: Track changes to DAG structure over time through the UI
- **DAG Bundling**: Package and distribute DAGs from Git, NFS, or other sources
- **Improved Diagnostics**: Understand when tasks were added/removed and how structure evolved

Development Workflow

The typical DAG creation workflow follows these steps:

1. **Write DAG code** in Python with proper structure and dependencies
2. **Test locally** using Airflow's testing framework

Follow – Sagar Khare

3. **Apply linting** to ensure code quality standards
4. **Version control** using Git with proper CI/CD practices
5. **Deploy to DAG folder** for scheduler discovery and parsing

---

Chapter 3: DAG Scheduling Phase

Understanding DAG Scheduling

Scheduling determines when a DAG runs. Airflow separates the logical concept of when a DAG should run from actual execution time through the data interval[1]:

Data Interval

Each DAG run has a data interval (start to end date) that defines what data that run processes. This enables:

- **Backfilling**: Re-run DAGs for historical periods without recreating the current state
- **Partition-based processing**: Each run processes data for its interval
- **Consistency**: Same interval always produces same results

Schedule Interval Configuration

```
with DAG(
dag_id='scheduled_dag',
schedule_interval='@daily', # Run daily
start_date=datetime(2025, 2, 18),
catchup=False # Don't backfill past runs
) as dag:
pass
```

Common Schedule Interval Options[4]:

- `@hourly`: Run every hour
- `@daily`: Run every day at midnight UTC
- `@weekly`: Run every Monday at midnight UTC
- `0 9 * * *`: Custom cron expression (9 AM daily)
- `timedelta(hours=2)`: Run every 2 hours

Advanced Scheduling Patterns

Dataset-Based Scheduling (Airflow 3.0+)

Modern Airflow supports event-driven scheduling through Datasets, evolving into Data Assets[3]:

```
from airflow import DAG
from airflow.datasets import Dataset

raw_data = Dataset('s3://bucket/raw_data/')

with DAG(
dag_id='downstream_dag',
schedule=[raw_data] # Trigger when dataset updates
) as dag:
pass
```

SLA and Timeout Management

Ensure DAGs meet Service Level Agreements:

```
with DAG(
dag_id='sla_dag',
sla=timedelta(hours=2), # Complete within 2 hours
default_args={'execution_timeout': timedelta(hours=1)}
) as dag:
pass
```

Scheduler Components

The Airflow Scheduler handles the scheduling layer with these responsibilities[2]:

- **DAG Parsing**: Discovers and parses DAG files every `min_file_process_interval` (default: 30 seconds)
- **Instance Creation**: Creates DAG runs at scheduled times
- **Dependency Resolution**: Determines when tasks can execute based on dependencies
- **Queue Management**: Sends ready tasks to executors

Chapter 4: DAG Execution Phase

Execution Models

Airflow supports multiple execution models depending on infrastructure requirements[2]:

Sequential Executor

Runs tasks one at a time on the scheduler machine:

airflow.cfg

executor = SequentialExecutor

Best for: Development, small workloads, testing

Follow – Sagar Khare

Celery Executor

Distributes tasks across worker nodes using a message broker (Redis/RabbitMQ):

airflow.cfg

executor = CeleryExecutor
broker_url = redis://localhost:6379

Best for: Large-scale deployments, complex dependencies, high throughput

Kubernetes Executor

Spins up Kubernetes pods for each task, ideal for cloud-native deployments:

airflow.cfg

executor = KubernetesExecutor

Best for: Cloud-native architectures, auto-scaling requirements, containerized workloads

Task Execution Workflow

When a DAG Run is triggered, tasks execute through this lifecycle:

1. **Task Queuing**: Ready tasks are queued based on dependencies and resource constraints
2. **Task Pickup**: Workers/executors pick up queued tasks
3. **Task Execution**: Operator executes the task logic (Python code, SQL, Bash, etc.)
4. **State Transition**: Task transitions through states: QUEUED → RUNNING → SUCCESS/FAILED
5. **Retry Logic**: Failed tasks are retried according to `retries` and `retry_delay` settings

Handling Failures and Retries

Robust error handling is critical for production DAGs:

```
default_args = {
'retries': 3,
'retry_delay': timedelta(minutes=5),
'retry_exponential_backoff': True,
'max_retry_delay': timedelta(minutes=30)
}

with DAG('resilient_dag', default_args=default_args) as dag:
task = PythonOperator(
task_id='critical_task',
```

```
python_callable=process_data,
on_failure_callback=send_alert # Custom failure handling
)
```

Performance Optimization

Optimize DAG execution for scalability[2][3]:

- **Parallelization**: Design DAGs with parallel branches to maximize throughput
- **Resource Limits**: Use `pool` and `queue` parameters to manage resource allocation
- **Task Duration Tracking**: Monitor and optimize long-running tasks
- **Intelligent Backfills**: Use AIP-78 scheduler-managed backfills for historical data processing
- **Late Binding**: Defer parameter resolution until execution for dynamic workflows

---

Chapter 5: DAG Monitoring and Observability

Monitoring Framework

Effective monitoring separates core Airflow system health from individual DAG performance[4]:

Core Airflow System Metrics

Monitor these critical components:

- **Scheduler**: Heartbeat, DAG parsing duration, task scheduling latency
- **Metadata Database**: Query performance, connection pool usage
- **Triggerer**: Event processing latency, concurrency limits
- **Executors/Workers**: Task throughput, queue depth, resource utilization
- **Web Server**: Response times, API latency

DAG-Level Metrics

Track performance for each DAG:

- **Task Success/Failure Rates**: Monitor failure patterns and trends
- **Task Duration**: Alert on unexpected increases indicating performance degradation
- **DAG Run Duration**: Ensure SLAs are met
- **Data Quality**: Task output validation and completeness checks
- **Backlog**: Monitor task queue depth during high-load periods

Logging and Centralized Log Management

Airflow provides comprehensive logging; centralize for better debugging:

airflow.cfg

```
remote_logging = True
remote_log_conn_id = s3_connection
remote_base_log_folder = s3://airflow-logs/
```

Integrate with ELK Stack or CloudWatch for searchable, audit-trail logs:

- Include `task_id`, `dag_id`, `execution_date` in all custom logs
- Implement log retention policies for compliance
- Correlate logs across schedulers, workers, and pipelines for root cause analysis

Alerting and Notifications

Implement multi-layer alerting:

```
def send_alert(context):
"""Send notification on task failure"""
task = context['task']
exception = context.get('exception')
# Send to Slack, PagerDuty, email, etc.

default_args = {
'on_failure_callback': send_alert,
'email_on_failure': True,
'email': ['ops@company.com']
}
```

SLA Management

Service Level Agreements define expectations for DAG completion:

```
with DAG(
dag_id='sla_monitored_dag',
sla=timedelta(hours=2),
default_view='tree'
) as dag:
task = PythonOperator(
task_id='process_task',
python_callable=process,
sla=timedelta(hours=1) # Task-level SLA
)
```

Observability Tools and Integration

Metrics Collection:

Follow – Sagar Khare

- StatsD/Prometheus: Export Airflow metrics
- Datadog, New Relic: Commercial monitoring platforms
- Grafana: Visualization and dashboarding

Data Lineage Tracking:

- OpenLineage: Standard for tracking data flow across pipelines
- Marooches: Understanding data dependencies and impact analysis

Web UI Features:

- **Tree View**: Visualize DAG run and task history
- **Graph View**: See task dependencies and status
- **Gantt Chart**: Monitor task timing and identify bottlenecks
- **Logs View**: Access detailed task execution logs

Chapter 6: Best Practices and Production Patterns

Infrastructure and Deployment

Automation and Immutability[3]:

- Automate infrastructure provisioning with Infrastructure-as-Code (Terraform, CloudFormation)
- Use ephemeral executors (e.g., Kubernetes) for cost efficiency
- Version control all DAG code in Git
- Implement CI/CD pipelines for automated testing and deployment

High Availability Configuration:

airflow.cfg

```
[core]
executor = CeleryExecutor
broker_url = redis://redis:6379
load_examples = False
load_default_connections = False

[scheduler]
scheduler_heartbeat_sec = 5
catchup_by_default = False
```

Code Quality and Testing

Follow – Sagar Khare

- Use `airflow dags test` for local validation
- Implement unit tests for custom operators and hooks
- Apply static code analysis (pylint, flake8)
- Conduct code reviews before production deployment

Performance Tuning

- **Monitor scheduler health**: Ensure DAG parsing completes quickly
- **Optimize task dependencies**: Minimize serialization overhead
- **Resource pooling**: Use Airflow pools to limit concurrent tasks
- **Database maintenance**: Regular cleanup of old metadata records

Security Considerations

- Secure connection strings in Airflow Connections (encrypted at rest)
- Use role-based access control (RBAC) in Web UI
- Enable remote logging to prevent log tampering
- Audit access and modifications to DAG definitions

Conclusion

The Airflow DAG lifecycle from Creation through Scheduling, Execution, and Monitoring represents a complete journey from pipeline design to operational excellence. Success requires attention to best practices at each phase:

Creation: Design idempotent, independent tasks with proper versioning
Scheduling: Leverage time-based and event-driven scheduling for flexibility
Execution: Choose appropriate executors and implement robust error handling
Monitoring: Build comprehensive observability for proactive issue detection

Modern Airflow (3.0+) introduces powerful features like DAG versioning, Data Assets, and event-driven scheduling that enable organizations to build enterprise-grade data platforms. By following the patterns and best practices outlined in this document, data engineers can create scalable, reliable, and maintainable workflows that power data-driven decision-making.

Follow – Sagar Khare